Ledgerly — Architecture & Database Design

Personal finance & shared expense tracker · v1.0 · July 2026 · No AI/LLM dependencies anywhere

1 · Product shape & phasing

Ledgerly launches as a single-user personal finance app and grows into a shared-expense platform (Splitwise-style) without schema rewrites. Everything below is deterministic and rule-based — no LLM, no external ML services, in any phase.

- **Phase 1 — Personal:** accounts, transactions, transfers, budgets, bills, recurring rules, reports, analytics, receipts.
- **Phase 2 — Shared:** friends, groups, splits, settlement engine, notifications, invitations.
- **Phase 3 — Data & polish:** Monito import wizard (generic migration engine), exports (PDF/CSV/XLSX), audit log, advanced analytics.

The critical decision enabling this: **every table carries ownership from day 1** (`userId` or group linkage), and transactions are modeled so a personal expense and a shared expense are the same row with an optional split attached — Phase 2 adds tables, it never alters Phase 1 rows.

2 · Stack decisions & trade-offs

Layer	Choice	Why / trade-off
Frontend	Next.js 15 (App Router), TypeScript, Tailwind, shadcn/ui, Framer Motion	One codebase, RSC for fast dashboards, shadcn gives owned (not installed) components matching the Glass & Ink design.
Backend	Next.js API routes + server actions	NestJS is overkill for one developer; revisit only if a mobile app needs a standalone API. Keep domain logic in `src/server/services` so it could move to NestJS untouched.
Database	Supabase Postgres + Prisma ORM	Prisma for typed queries/migrations; Supabase for hosting, RLS, storage. Prisma connects via the pooled connection string (PgBouncer).
Auth	**Better Auth** (recommended)	Self-hosted, no per-MAU pricing (vs Clerk), first-class Prisma adapter, owns the user table in your DB. Clerk is faster to ship but couples your user identity to a vendor.
Charts	Recharts	Declarative, fine for bar/line/donut. Swap per-chart to visx only if the expense heatmap needs it.
Storage	Supabase Storage	Receipts in a private bucket, path `receipts/{userId}/{txId}`, served via short-lived signed URLs.
Jobs	Vercel Cron → internal route	Daily 00:30 IST: materialize recurring transactions, generate bill instances, fire notification rules. Idempotent (keyed on rule id + period).

3 · Application architecture

Three strict layers. UI never touches Prisma directly; services never touch HTTP. This is what keeps the codebase beginner-friendly and portable.

```
src/
├─ app/                    # routes (App Router)
│  ├─ (app)/dashboard, transactions, accounts, budgets,
│  │        bills, shared, analytics, settings, import/
│  ├─ (auth)/sign-in, sign-up/
│  └─ api/                 # route handlers: REST-ish JSON + cron
├─ components/             # ui/ (shadcn), charts/, forms/, layout/
├─ server/
│  ├─ services/            # domain logic: transactions.ts, budgets.ts,
│  │                       #   settlements.ts, recurring.ts, import/
│  ├─ auth.ts              # Better Auth config
│  └─ db.ts                # Prisma client (singleton)
├─ lib/                    # money.ts (paise math), dates.ts, search-parser.ts
├─ validators/             # zod schemas — single source for API + forms
└─ prisma/schema.prisma
```

- **Money is integer paise** (`BigInt` in Prisma, ₹1 = 100). Never floats. Formatting via `Intl.NumberFormat('en-IN')` at the edge only.
- **Account balances are derived**, not stored: a materialized running balance is maintained per account inside the same DB transaction that writes a Transaction row, and can always be rebuilt from `openingBalance + Σ ledger`. Corruption-proof and auditable.
- **Optimistic UI** via TanStack Query mutations with rollback; pagination is cursor-based (id + date) everywhere.

4 · Database schema (Prisma)

Key modeling decisions, then the schema. Enums are abbreviated for readability.

1. **One `Transaction` table** for expense / income / transfer (a `type` enum + nullable `toAccountId` for transfers). Reports, search and import all query one table.
2. **Splits attach to transactions.** A shared expense is a normal Transaction plus `ExpenseSplit` rows (one per participant, storing owed paise). Works for equal / percent / exact / ratio — the split method is metadata; the stored truth is always exact paise per participant, which must sum to the total (DB check constraint).
3. **Participants can be non-users.** `Participant` rows have a nullable `linkedUserId` — you can split with "Rohan" before Rohan ever signs up; when he accepts an invitation his participant record links to his account and history follows him.
4. **Categories are per-user rows** seeded from a default set at signup (not a global table), so renames/custom categories never collide between users.

```
model User {
  id            String   @id @default(cuid())
  email         String   @unique
```

```
  name          String
  currency      String   @default("INR")
  createdAt     DateTime @default(now())
  accounts      Account[]
  transactions  Transaction[]
  categories    Category[]
  // budgets, bills, recurringRules, participantsOwned, notifications…
}

enum AccountType { BANK CASH WALLET CREDIT_CARD INVESTMENT }

model Account {
  id             String      @id @default(cuid())
  userId         String
  user           User        @relation(fields: [userId], references: [id])
  name           String      // "HDFC Savings"
  type           AccountType
  bankName       String?
  openingBalance BigInt      @default(0)   // paise
  balance        BigInt      @default(0)   // maintained in-tx, rebuildable
  color          String?
  icon           String?
  isArchived     Boolean     @default(false)
  @@index([userId])
}

enum TxType { EXPENSE INCOME TRANSFER }

model Transaction {
  id            String    @id @default(cuid())
  userId        String    // who recorded it
  type          TxType
  amount        BigInt    // paise, always positive
  accountId     String?   // source account (null: paid by someone else)
  toAccountId   String?   // TRANSFER only
  categoryId    String?
  merchant      String
  occurredAt    DateTime
  notes         String?
  location      String?
  isRecurring   Boolean   @default(false)
  recurringRuleId String?
  importBatchId String?
  groupId       String?   // shared context, optional
  paidByParticipantId String? // null ⇒ paid by owner
  splits        ExpenseSplit[]
  tags          TransactionTag[]
  receipts      Receipt[]
  deletedAt     DateTime? // soft delete → undo
  @@index([userId, occurredAt(sort: Desc)])
  @@index([userId, categoryId]) @@index([userId, accountId])
  @@index([userId, merchant])
}
```

```
model Category {
  id        String  @id @default(cuid())
  userId    String
  name      String  // seeded defaults + custom
  kind      TxType  // EXPENSE or INCOME
  parentId  String? // subcategories
  icon      String?
  color     String?
  @@unique([userId, name, kind])
}

model Budget {
  id         String  @id @default(cuid())
  userId     String
  categoryId String? // null ⇒ overall budget
  accountId  String? // account-scoped budgets
  period     BudgetPeriod // WEEKLY MONTHLY YEARLY
  limit      BigInt
  alertAt    Int     @default(80) // % threshold notification
  @@unique([userId, categoryId, accountId, period])
}

model Bill {
  id          String   @id @default(cuid())
  userId      String
  name        String
  amount      BigInt
  categoryId  String?
  dueDate     DateTime
  recurringRuleId String? // auto-regenerated after payment
  status      BillStatus @default(UPCOMING) // UPCOMING PAID OVERDUE
  paidTxId    String?  // links the payment transaction
}

model RecurringRule {
  id        String   @id @default(cuid())
  userId    String
  cadence   Cadence  // DAILY WEEKLY MONTHLY QUARTERLY YEARLY
  interval  Int      @default(1)
  nextRunAt DateTime
  endsAt    DateTime?
  template  Json     // transaction/bill payload to materialize
  kind      RuleKind // TRANSACTION or BILL
  @@index([nextRunAt])
}

model Tag { id String @id @default(cuid()); userId String; name String
  @@unique([userId, name]) }
model TransactionTag { txId String; tagId String; @@id([txId, tagId]) }

model Receipt {
  id String @id @default(cuid()); txId String; userId String
```

```
  storagePath String; mimeType String; sizeBytes Int
  uploadedAt DateTime @default(now())
}
```

Phase 2 — sharing, groups, settlements:

```
model Participant {           // a person you split with
  id           String  @id @default(cuid())
  ownerId      String  // whose contact list
  displayName  String
  linkedUserId String? // set when they join via invitation
  @@unique([ownerId, linkedUserId])
}

model Group {
  id String @id @default(cuid()); name String; createdById String
  members GroupMember[]
}
model GroupMember {
  groupId String; participantId String
  role    GroupRole @default(MEMBER) // OWNER ADMIN MEMBER
  @@id([groupId, participantId])
}

model Invitation {
  id String @id @default(cuid()); email String; invitedById String
  participantId String; token String @unique; status InviteStatus
  expiresAt DateTime
}

model ExpenseSplit {              // truth = exact paise per head
  id            String @id @default(cuid())
  txId          String
  participantId String? // null ⇒ the owner's own share
  owedAmount    BigInt  // Σ per tx must equal tx.amount (CHECK)
  method        SplitMethod // EQUAL PERCENT EXACT RATIO CUSTOM
  isSettled     Boolean @default(false)
  @@index([participantId, isSettled])
}

model Settlement {
  id        String   @id @default(cuid())
  payerParticipantId String  // who paid
  payeeParticipantId String  // who received
  amount    BigInt
  method    SettleMethod // UPI CASH BANK
  settledAt DateTime @default(now())
  note      String?
}

model Notification {
  id String @id @default(cuid()); userId String
  kind NotifKind // BUDGET_EXCEEDED BILL_DUE SETTLE_REMINDER
                 // RECURRING_CREATED MONTHLY_SUMMARY LARGE_EXPENSE
  payload Json; readAt DateTime?; createdAt DateTime @default(now())
  @@index([userId, readAt])
}
```

Phase 3 — import engine & audit:

```
model ImportBatch {
  id String @id @default(cuid()); userId String
  source String        // "monito" | "csv" | "splitwise" | …
  fileName String; importedCount Int; skippedCount Int
  errors Json?; status ImportStatus // PREVIEW COMMITTED UNDONE
  createdAt DateTime @default(now())
  // undo = soft-delete all tx where sc-camel-import-batch-id = id
}

model ImportMapping {   // remembered column + category mappings
  id String @id @default(cuid()); userId String; source String
  columnMap Json        // {"Date":"occurredAt","Note":"merchant",…}
  categoryMap Json      // {"Eating Out":"Food", …}
  @@unique([userId, source])
}

model AuditLog {
  id String @id @default(cuid()); userId String; action String
  entity String; entityId String; before Json?; after Json?
  at DateTime @default(now())
  @@index([userId, at(sort: Desc)])
}
```

5 · Deterministic engines (no AI, by design)

- **Settlement engine:** compute net balance per participant from unsettled splits minus settlements; minimize transactions with the greedy max-debtor→max-creditor algorithm (optimal count ≤ n−1). Pure function in `services/settlements.ts`, unit-tested against paise-rounding edge cases (remainder paise assigned to the payer).
- **Search parser:** tokenizer → matchers against the user's own merchants, categories, accounts, tags, date phrases ("last month", "in March", "this year"), amount ranges ("above ₹500"), and type keywords — compiled to a Prisma `where` object. Falls back to Postgres full-text search on merchant + notes. "Show my UPI expenses" = account-type matcher, zero AI.
- **Auto-categorization:** a per-user `merchant → category` rule table, seeded with a common-merchant dictionary (Swiggy→Food, BESCOM→Electricity…) and updated every time the user re-categorizes. Deterministic and self-improving.
- **Import engine:** adapter pattern — `ImportAdapter` interface (`detect · parse · mapColumns · validate`) with a MonitoAdapter first; Splitwise/bank-statement adapters plug in later. Wizard flow: upload → auto-detect columns (header heuristics + value-shape scoring) → mapping UI (persisted to ImportMapping) → preview with duplicate detection (hash of date+amount+merchant±1 day) → row-level skip/edit → commit as one ImportBatch → one-click undo.

- **Recurring engine:** cron reads `RecurringRule.nextRunAt ≤ now`, materializes the row inside a transaction, advances `nextRunAt`. Unique constraint on (ruleId, period) makes re-runs idempotent.

6 · Security & performance

- **Authorization:** every service function takes the session user and scopes queries by `userId`; Supabase **RLS policies mirror the same rules** as defense-in-depth (shared rows readable by group members only).
- **Validation:** zod schemas shared between forms and API; all money parsed to paise at the boundary. Rate limiting via Upstash on auth + import routes. File uploads: type/size allowlist, private bucket, signed URLs.
- **Audit:** writes to money-bearing tables append an AuditLog row (before/after JSON).
- **Performance:** covering indexes above match every dashboard query; monthly aggregates served from a `month × category × account` rollup view (refreshed on write, cheap at personal scale); cursor pagination; charts lazy-loaded; receipts via `next/image`.

7 · Build order

Milestone	Scope
M1 · Skeleton	Repo, Prisma schema (all phases — migrate once), Better Auth, seed script, app shell + dark mode matching the prototype.
M2 · Ledger core	Accounts CRUD, transactions (expense/income/transfer), balance maintenance, transaction list + filters + search parser.
M3 · Dashboard	Stat cards, cash flow, category donut, attention strip, budgets + alerts, bills + recurring engine (cron).
M4 · Analytics	Trends, merchant analysis, balance trend, reports + CSV/XLSX/PDF export, receipts.
M5 · Shared	Participants, groups, invitations, splits, settlement engine, notifications.
M6 · Import	Monito adapter + wizard, import history, undo. Then real-data migration.